ssl-mgr Documentation

Release 8.0.0

Gene C

Sep 07, 2026

SSL-MGR DOCUMENTATION

CHAPTER

ONE

SSL-MGR

sslm-mgr is a certificate management tool that helps automate key/certificate creation and renewal. It handles multiple certificates for one or more domains.

See the *Docs* directory for the complete PDF documentation.

A key pair is comprised of a private *key* and a *certificate* holding the public key.

1.1 Key Features

- Create new key/cert pairs
- Renew existing ones.
- Generate key pairs using Certificate Signing Request (CSR) for maximum control.
- Supports http-01 and dns-01 acme challenges.
- Support of dns-persist-01 planned for 2026 after available in Letsencrypt.
- Outputs files for acme DNS-01 authentication with appropriate DNS TXT records.

  These files can be $INCLUDED in the apex domain zone file, making updates straightforward.
- Optional support to output DANE TLSA file for each Apex domain.

  These files can be $INCLUDED in apex domain DNS Zone file.
- Uses certbot in manual mode to communicate with letsencrypt, account tracking etc.
- Processes multiple domains - each domain can have multiple certs.

  For example making separate certs for web and email servers.

1.2 Recent Updates

8.0.0

- Use meson / meson-python for build and package management.
- Code review

1.3 Overview

Secure communications often use private / public key pairs. The public key for many cases is contained within a certificate (*cert*) which is signed by some certificate authority (*CA*) and has an expiration date. While certs have expiration date, keys (private or private) do not.

There are several known and *approved* CAs, such as Letsencrypt, whose own certificates are known and trusted. Browsers for example have trust in them. When a certificate is presented that has been signed by one of these CAs, then the browser trusts that certificate as well.

Obtaining a certificate from Letsencrypt and other CAs is done using the Automated Certificate Management Environment (ACME) protocol.

See *Acme Challenge* for some background how CA certificates are validated.

Web and mail servers both use key/certs. Web servers have browsers and other clients and mail servers (mostly) have other mail servers as clients.

Letsencrypt is a CA service that provides signed certificates for these kinds of uses. Their certificates have moderately short (and getting shorter) expirations and thus need to be renewed periodically.

I wrote *ssl-mgr* with 3 goals. Specifically to:

- Simplify certificate management - (i.e. automatic, simple and robust)
- Support *dns-01* acme challenge with Letsencrypt (as well as *http-01*)
- Support *DANE TLS*

Creating and reenewing key/certs can be tricky. *ssl-mgr* strives to make things robust, complete, clean and as simple to use as possible. Under the hood, make it sensible (do the *right* thing) and automate wherever feasible.

A good tool does things correctly while making using the tool as straightforward and simple as possible; but no simpler.

In practical terms, there are only 2 key commands needed with *sslm-mgr*:

- **renew** - creates new certificate(s) in *next* : current certs remain in *curr*.
- **roll** - moves *next* to be the new *curr*.

The configuration files provide all the relevant information about the domains and certificates needed. After the configuration files are set up with the appropriate information then these commands can be run out of cron:

- renew, wait, roll.

Clean and simple.

Rolling of certs is strictly only needed when certs are advertised via DNS (for example) and some time is required for DNS caches to flush through. This is true for DANE, since it provides certs via DNS *TLSA* records.

In the first roll step, both old and new keys are advertised and they are kept available (in DNS) for an appropriate period of time; typically long enough for DNS info to propogate and DNS servers to update.

In the second, and last, roll step, DNS is updated to advertise only the new certs.

Changing to new certs without rolling can be problematic if some DNS servers still point to the older certs.

Without any loss of generality we can always renew and roll. If a roll is not needed, then the roll wait time can be adjusted to a small value, or even to 0.

N.B.

DNSSEC is required for DANE otherwise its not needed. However, we still recommend using DNSSEC and made available the tool we use to simplify DNS/DNSSEC management[1].

Important

- When activating DANE for the first time it's important to only include DANE TLSA records after the *roll*. See *DANE First Use - Caution* for more detail.

[1] dns_tools : https://github.com/gene-git/dns_tools

DANE can use either self-signed certs or *known* Certificate Authority (CA) signed certs. *ssl-mgr* makes it straightforward to create self-signed certs as well.

The recommended way for creating and using your own certs is to have your own self-signed CA which is then used to sign an *intermediate* certificate. The *intermediate* cert is what is then used to sign other certificates. This is the same model used by well-known CAs as well. So it keeps it clean and simple to follow the same model.

While mail may use your own CA signed certs, , in practice, it is safer to use CA signed certs for to reduce the chance of delivery problems in the event a mail server requires a CA chain of trust.

Therefore we therefore using CA signed certificates and publishing DANE TLSA records using those certificiates. Each MX will have its own TLSA record.

While DANE can be used for other TLS services, notably https, in practice it is only used with email.

1.4 Cert Information

sslm-mgr –status provides a convenient summary of all managed certificates along with their expiration and time remaining time before the next renewal.

Sample output of *sslm-mgr –status –verb example.com:web-ec-self*:

```
web-ec-self
    curr         : expires: 2026-07-06 12:07:39+00:00 (  89 days + 23:59:07)
                 : issued : 2026-04-07 12:07:39+00:00 (   0 days + 00:00:52 ago.
↪ 90 day cert)
          issuer : CN=MySub-ec O=Example IT Department
         subject : CN=example.com
          pubkey : pubkey-secp384r1
            sans : ['example.com', 'www.example.com']
       sig_algo : ecdsa-with-SHA384
```

The last two lines are included when *–verb* option is used.

1.5 File Information

While *sslm-mgr* interacts with the internal data, there are times when its useful to find information about a certificate file. The file can be a private key, a certificate (or chain of certs) or CSR.

sslm-info <filename> displays information about it's contents.

sslm-verify checks whether a cert is valid. Sample output:

```
--------------------
Certificate ** Verified **
Certificate Info:
Expires  : 2026-06-23 17:50:05+00:00 (  77 days + 05:24:38)
Issued   : 2026-03-25 17:50:06+00:00 (  12 days + 18:35:20 ago. 90 day cert)
Issuer   : CN=YE2,O=Let's Encrypt,C=US
Pubkey   : pubkey-secp384r1
Sig algo : ecdsa-with-SHA384


--------------------
Chain Info:
```

(continues on next page)

(continued from previous page)

```
Expires  : 2028-09-02 23:59:59+00:00 ( 879 days + 11:34:32)
Issued   : 2025-09-03 00:00:00+00:00 ( 216 days + 12:25:26 ago. 1096 day cert)
Issuer   : CN=Root YE,O=ISRG,C=US
Subject  : CN=YE2,O=Let's Encrypt,C=US
Pubkey   : pubkey-secp384r1
Sig algo : ecdsa-with-SHA384

Expires  : 2032-09-02 23:59:59+00:00 (2340 days + 11:34:32)
Issued   : 2025-09-03 00:00:00+00:00 ( 216 days + 12:25:26 ago. 2557 day cert)
Issuer   : CN=ISRG Root X2,O=Internet Security Research Group,C=US
Subject  : CN=Root YE,O=ISRG,C=US
Pubkey   : pubkey-secp384r1
Sig algo : ecdsa-with-SHA384

Expires  : 2032-09-02 23:59:59+00:00 (2340 days + 11:34:32)
Issued   : 2025-09-03 00:00:00+00:00 ( 216 days + 12:25:26 ago. 2557 day cert)
Issuer   : CN=ISRG Root X1,O=Internet Security Research Group,C=US
Subject  : CN=ISRG Root X2,O=Internet Security Research Group,C=US
Pubkey   : pubkey-secp384r1
Sig algo : sha256WithRSAEncryption
```

1.6 Git Repo

Please Note:

All git tags are signed with the arch@sapience.com (public) key that is available via WKD or for download from https://www.sapience.com/tech. Add the key to your package builder openpgp keyring.

The key is also included in the Arch package and the source= line with *?signed* at the end of the line can be used to verify the git tag. You can also manually verify the signature as usual with *git verify*.

1.7 ssl-mgr: Details

More details about *ssl-mgr* tools are available in the sections *Acme Challenge*, *Background*, *Renewing & Rolling*, *DANE*, *Configs: Getting Started* and *Using the Tools : Details*.

CHAPTER

TWO

BACKGROUND

2.1 Groups & Services

To keep things organized, data is partitioned into groups and services.

There are two types of groups:

- Certificate Authorities (CAs)
- Apex Domains.

A CA can be an external Authority such as Letsencrypt or an internal / local one.

A local CA is structured similarly to an external one by having a self-signed root certificate along with a companion intermediate cert(s) that cert is signed by the root cert. Intermediate certs are then used to sign any requested certificates.

Certificates are typically used by a server such as a web or email server. Each certificate is defined by a *service*. Certificates carry an apex domain (e.g. example.com) but may have additional subdomains (sub1.example.com, sub2.example.com). The full list of all these domains in a certificate is stored under Subject Alternative Names or SANS.

An apex domain can have multiple certificates, where each certificate includes all the subdomains along with the apex domain. All the desired details for each certificate is provided in a *service* config file.

2.1.1 Certificate Authorities

The job of a CA is to take a Certificate Signing Request (CSR) and return a signed cert.

- Internal CA
  - Self-signed root certs are used solely to sign local intermediate certs.
  - Local intermediate certs are used to sign certs. The intermediates are signed by a local self signed root.

  Using internal certs are a good starting point when getting set up and exploring *ssl-mgr*.
- Letsencrypt CA

  When ready, using their test server, which is more generous with limits, is a good way to prepare for the production. LE's test server is invoked by using the *-t* option. The *–dry-run* option may be helpful too.

  When all is working as desired, drop the test option and you're ready to go into production.

2.1.2 Apex Domains

An Apex domain is the *main* part of the domain with it's own DNS authority.

If *example.com* has a DNS SOA record, then it would be the apex domain and any of it's subdomain, such as *foo.example.com* would be a part of that apex domain. Whenever we deal with DNS, we (almost) always deal with the apex domain.

Each apex domain is a *group* and may have 1 or more certificates where each certificate is associated with 1 service.

2.1.3 Services

Each service gets 1 certificate.

An apex domain may want/need different certs for different services. Each service has one certificate.

An apex domain, for example, may have a mail service and a web service. Each of these has it's own unique cert. Now, mail may use 2 certs, perhaps elliptic curve and RSA, then we would simply have 2 services for mail. In this case lets call them *mail-ec* and *mail-rsa* and lets call the web service *web-ec*. Its good to name services in a way thats useful for administrator - it has no significance to the code other than the name must be a good filename so cannot contain / etc.

In the same vein, for self signed CA certs, we have 2 items - a *root* cert and an *intermediate* cert where each belongs the special group *ca*. Again, each of these is a separate service.

Since each service has its own certificate, each has its own X509 name which describe what it is. This includes things like Common Name, Alternative Names and organization. In this case it includes info about the keys to be used and which entity is provides the signed certificate.

Each service has it's information provided by a service file. It has all the information needed to create keys and CSRs as well as certs. This include key type, various *name* fields along with which CA should be used. The *name* fields are essentially *x509* Name[1] fields. These include things like Common Name, Organization and so on.

CSR (certificate signing request) contains the *subject* organiziation (thats the apex domain org) information along with the public key. The private key is kept in a separate file. The CSR is sent to the CA which, all being well, returns a (signed) certificate.

The resulting cert and certificate chain(s) are kept together with the key and CSR files. A cert is signed by the *Issuer* and in addition to the signature contains the public key. The *chain* file contains the public key and x509 Name of the certificate issuer.

There are a couple of tools provided (*sslm-verify* and *sslm-info*) that make it easy to validate a certificate or display information about it. *sslm-info* works on all the *sslm-mgr* outputs : keys, csrs, certs, chains, fullchains and bundles.

2.2 Key/Cert Files

- CSR (certificate signing request)

  Each certificate for is generated from its CSR which contains the public key. Public key is generated from the private key so there is no need to save a public key.

  A CSR is always used make a cert. This provides control as well as consistency across CAs, be they self or other. The public key is in the CSR and also in the certificate provided and signed by the CA. We support both RSA and Elliptic Curve (EC) keys. EC is strongly preferred. In fact, while RSA keys are still used they are only needed by ancient client software for browsers and email. That said, RSA is still in common use for DKIM[2] signing for some reason. We DKIM sign outbound mail with both RSA and EC.

[1] x509 Name https://en.wikipedia.org/wiki/X.509
[2] DKIM -> https://datatracker.ietf.org/doc/html/rfc6376

- Cert

  Each cert contains the public key which is signed by the CA. It carries the *subject* apex domain name along with 'subject alternative names' or SANS. SANS allow a certificate to contain multiple domain or subdomain names. The *issuer*, which signed the certificate, has it's name in the cert as well. Name in this context is an X509 name meaning, common name, organization, organization unit and so on.

- Certificate chains

  - **chain** = CA root cert + Signing CA cert

    Signing CA cert is usually the CA Intermediate cart(s) Note that the root cert may or may not be included by CAs other than LE For those client chain = signing ca

  - **fullchain** = Domain cert + chain

  - **bundle** = priv-key + fullchain.

    A bundle is just a chain made of the private key plus the fullchain. This is preferred by postfix[3].

- Private key

  Also called simply the *key*. It is stored in a file with restricted permissions. The companion public key can be generated from the private key. By always generating the public key from the private key, they are guaranteed to remain consistent.

Key, CSR and certificate files are stored in the convenient PEM format. Certificates use X509.V3[4] which provides for *extensions* such as SANS which are critical to have. CSR files use *PKCS#10*[5] which can carry the same set of X509 extensions.

[3] Postfix TLS -> https://www.postfix.org/postconf.5.html#smtpd_tls_chain_files
[4] X509 V3 -> https://datatracker.ietf.org/doc/html/rfc5280
[5] PKCS#10 CSR -> https://www.rfc-editor.org/rfc/rfc2986

CHAPTER

THREE

ACME CHALLENGE

Letsencrypt uses Automated Certificate Management Environment (ACME) and while our focus is on Letsencrypt there are other providers using ACME as well.

The protocol provides a mechanism for the CA to provide a signed certificate. The way *ssl-mgr* works is by creating a Certificate Signing Request (CSR) which is sent to Letsencrypt which returns a signed certificate. The CSR and the cert contain the public key. The companion private key is never shared.

Before the CA signs the certificate it validates that we have control over the Apex domain and any other subdomains that are part of the certificate. There a various ways the validation occurs.

One validation method uses a web server (the http-01 protocol) and another uses DNS server (dns-01). The validation generally involves the CA sending a secret which is then made available on the web server for the domain name being validated or available on the DNS server for the domain. After the CA has verified the correct secret has been posted on the web/dns server it then issues the certificate.

Using *DNS-01* to validate Letsencrypt acme challenges is done by adding the challenge TXT records to DNS, signing the zones (if using DNSSEC) and pushing them out, so that the signing CA can subsequently check those DNS records match appropriately and then they provide the requested cert.

While[1] can use either http-1 or dns-01; dns is preferred whenever possible.

There is a new acme protocol expected to be available in 2026 called dns-persist-01. This is similar to dns-01 but instead of requiring validation on every renewal, the idea is to have a long lasting, persistent, DNS record which can be re-used at renewal. This should simplify and speed up renewals.

Note that in order to update DNS, an appropriate tool is needed. We use dns_tools for that purpose.

To make this as smooth as possible, *ssl-mgr* should run on the DNS signing server. This allows files with DNS records, acme challenges and TLSA record files, to be written to accessible directories on same machine.

A possible future enhancement may allow the dns signing server to be remote.

Note that DNS server refresh is also done after any DANE TLSA records are updated.

[1] acme-val-challenge : https://letsencrypt.org/docs/challenge-types/

CHAPTER

FOUR

RENEWING & ROLLING

ssl-mgr keeps and manages 2 versions of every set of data. Each data set consists of certificates, keys, Certificate Signing Requests (CSRs) and dane tlsa records.

One version of the data holds the current (or *curr*) set and the other has the *next* set. *curr* are those currently in use while *next* are those that are on deck to become the next current set.

Key rolling is standard practice and should be familiar to those who have implemented *DNSSEC* for example. A *roll* provides a robust method to update keys/certs with new ones in a way that ensures nothing breaks.

For some uses, the current key/cert is advertised in DNS. After creating new keys/certs, DNS is then upated to advertise both the current and the newly created next ones.

An appropriate amount of time needs to pass with both current and next in DNS before doing the second phase of the *roll*. This gives the time needed for DNS servers to refresh. Once refreshed, the DNS servers have both the current and the next set of keys/certs.

After sufficient time, update a second time, after which only the new keys (the new current ones) are advertised in DNS.

A *roll* is required for *DNSSEC* keys (managed elsewhere) and for *DANE* which *ssl-mgr* is responsible for.

Without any loss of functionality and to keep things nice and simple, we require every update to use a key roll.

Again, a *roll* is required for *DANE TLS* but is not needed for things such as web server certificate update.

If you are not advertizing certificate info using DNS servers (e.g. DNSSEC, DANE) then there is no need have much or even any delay between making a new certificate using *-renew* and doing the *-roll*.

In this case, you can set the config variable *min_roll_mins* to 0 minutes. The default min roll time is 90 minutes. And if automating (via cron or similar) then you can also use do the *roll* immediately after the *renew* as well. In cron you could have roll set to run 1 minute after the renew.

If you have DANE, then it is important to have an appropriate delay after the *renew* before doing the *roll*.

Also, you are always have control and, should it ever be needed, you can do whatever you choose.

e.g. Using *-f* option with *sslm-mgr* will force things to happen (a roll or create new certs and so on.)

4.1 Curr & Next

These are kept in directories that contain different versions of the same set of files. Of course *next* has newer versions. For example for the group *example.com* and the service *web-ec* their output directories would be:

```
certs/example.com/web-rc/curr/
certs/example.com/web-rc/next/
```

In order of creation these are:

File	What
privkey.pem	the private key
csr.pem	certificate signing request
cert.pem	certificate
chain.pem	CA root + intermediate certs
fullchain.pem	Our cert.pem + CA chain
bundle.pem	Our privkey + fullchain
tlsa.rr	Optioanl Dane TLSA records
info	Contains date/time when next was rolled to curr (curr only)

Once config is setup, a cron/timer to run *renew* followed by *roll* 2 or 3 hours later should take care of everything. Can be run daily or weekly.

The *curr/next* directories are copied to the production directory, as specified in *conf.d/ssl-mgr.conf* by the variable *prod_cert_dir*. This directory can be copied to whatever servers need key/cert access.

4.2 Diffie-Hellman Parameters

The tool *sslm-dhparm* generates Diffie-Hellman parameters. This too can be added to be run by cron.

By default *sslm-dhparm* only generates new parameters if they are more than 120 days old, or absent. This can therefore be run weekly without issues.

Note: The new, preferred and now default DH parameters are based on the rfc_7919 *finite field* pre-defined named groups. The default is *ffdhe4096*. Pre-defined named groups only need to be generated once and *sslm-dhparm* will only generate them if absent.

Strictly these don't need to be in cron, but its convenient to have the program check and create DH parameters should the file be missing. May happen occasionally after adding new domain.

The 6 month default refresh, ony applies for non RFC-7919 params, and is recommended because it can be a bit time consuming to generate them. Actual time varies with key size.

When using a pre-defined named group (e.g. *ffdhe4096*), it is very quick to produce and tool simply checks if file exists without any age requirement. These are only created once.

Sample cron files are provided in the examples directory.

CHAPTER

FIVE

DANE

For DANE TLSA records, care must be taken to properly *roll* new keys. Key rolling ensures that the *next* key and the *curr* key are both advertised in DNS for some period. After some time the new key can be made *curr*. This waiting period should be long enough to provide sufficient time for all DNS servers to pick up both old and new new keys before DNS is changed to only show the new ones. It's reasonable to wait 2 x the DNS TTL or longer.

After that wait time, the new (*next*) keys can be then be made available as the new *curr* ones. Applications, mail really, can now use the new keys since the world has both sets of keys.

Then DNS servers can then be updated again, this time with just the new (now *curr*) keys in the TLSA records. DANE key roll is similar to key roll for DNSEC. DANE TLSA actually requires DNSSEC.

DANE was designed as an alternative to third party certificate authorities like letsencrypt which means its fine to used self signed or CA signed certs. While DANE could be used for web servers to date it is really only used for email.

The companion *dns_tools* package takes care of all our DNSSEC needs[1]:

And I recommend using it to simplify the DNS refresh needed for validating Letsencrypt acme challenges using *DNS-01* as well as for DANE TLSA. A DNS refresh means resign zones (when using DNSSEC) and then restarting the primary dns server.

DANE TLSA records contain the public key, or a hash of that key, and thus need to be refreshed whenever that key changes; this is the key roll. It also means that if the key is kept the same, then the TLSA records aren't changing[2]. *ssl-mgr* has an option to re-use the public key when certs are being renewed, and this allows the TLSA records to remain unchanged. In that case no key roll is needed until that key is changed. Some may find this useful.

It basically means using the same certificate signing request, CSR, to get a new cert. The CSR contains the public key associated with the private key. So if keys dont change CSR doesn't change either, and the same CSR can be re-used.

However, I find *ssl-mgr* makes it so simple to renew with new keys, that I don't see much point in reusing the old keys. Of course using new keys offers a security benefit.

Note that each MX for the mail domain will have a TLSA record as required by the standard.

5.1 DANE First Use - Caution

Important:

A cautionary note about the turning on DANE TLSA for the first time for an Apex domain.

Do not include dane TLSA records in DNS zone files, until after the *roll*.

[1] dns_tools : https://github.com/gene-git/dns_tools

[2] DANE can use either public key or the cert. Cert does change when it's reneweed even if the public key is unchanged. I believe pretty much everyone uses the public key not the cert in TLSA reords.

The very first time *DANE* is turned on, *tlsa* DNS records are created for *next*. The reason is, at this point there are no *tlsa* records for *curr* and the key/cert in *curr* is what mail server is using.

So if DNS is pushed out before the roll, these new *TLSA* records for *next* will be visible. However, the mail servers are still using the key/certs from *curr*. Mail servers attempting to send mail inbound to your servers will be checking for the certs being advertized which are from *next*, and these are not yet being used.

After the roll, it is fine to update the DNS zone file to include the DANE TLSA records, because the mail servers are now using the key/certs from *curr* as advertized by the *DANE* records.

Going forward after a renewal, there will be no issue. This is becuase both *curr* and *next TLSA* DNS records will both be advertized before the roll. Mail server is then using *curr* and there is a DANE records advertizing the *curr* cert.

So, after activating DANE the very first time for an Apex domain, hold off actually including those TLSA records in the zone file until after the *roll.* After the *roll, the mail servers are using the key/certs advertized in the DNS *DANE TLSA* records.

CHAPTER

SIX

CONFIGS: GETTING STARTED

ssl-mgr handles keys, certificate signing requests (CSR) and certs. It takes care of generating DANE TLSA DNS records should they be desired. It reloads/restarts specific servers whenever needed. Each server has defined dependencies which trigger restarts whenever those dependencies have changed.

For example, a web server may depend on one or more apex domain certificates and will be restarted when any of those certs change.

It needs external support tools such as zone signing for DNSSEC and restarting dns servers as well as reloading web or mail servers to ensure new certs are picked up. These are provided via the top level config file.

There is support for private/self-signed CAs and Letsencrypt CA.

The first order of business is to create the config files that describe what needs to be done. They specify everything needed to obtain one or more certificates for one or more domains.

There are 3 kinds of configs.

ssl-mgr.conf

This is the top level config which has the list of active apex domains and for each domain a list of services for that domain. Each service produces one certificate.

It also specifes the commands to be used to restart servers (e.g. web, mail) and where to write any acme challenge files (web or dns) that needed to validate domain control to the issuing CA.

Lastly it defines where the final (production) certificates are to be stored.

service configs

Each certificate to be issued for an apex domain for some service uses one configuration file. For example a service might be a mail server using elliptic curve algorithm where certificate is issued by Letsencrypt.

The service configs reside in a directory under its apex domain.

ca-info.conf

Information about each Certifice Authority is defined in this file.

Sample configs in examples/conf.d can be used as a template to get started.

CHAPTER

SEVEN

USING THE TOOLS : DETAILS

Once the config files are set up then the tools can be used to do the real work.

The primary tool for generating and managing certificates is *sslm-mgr*. As usual, help is available with the *-h* or *–help* option.

The 2 most frequently used options are *renew* and *roll*.

There are some additional options that give a bit more control, in case its ever needed. These are discussed in detail below. For example the *-f* or *–force* option does what the name suggests. So

```
sslm-mgr -f -renew
```

will force a cert to be renewed even if its not yet time to do so.

There is also *dev* mode. This provides access to some lower lever tasks. This should seldom, if ever, be needed. In case you do, it is activated by by adding *dev* command as the first argument.

For example to get help for dev mode:

```
sslm-mgr dev -h
```

The tools in the package include:

Tool	Purpose
sslm-auth-hook	internal - used with certbot's manual hook option
sslm-dhparm	generates Diffie Hellman paramater file(s)
sslm-info	displays info about cert.pem, csr.pem, chain.pem, privkey.pem, etc
sslm-mgr	primary tool for certificate management
sslm-verify	verifies any cert.pem file using the public key from chain.pem

7.1 sslm-mgr options

As already mentioned, once configs are set up and running, then all that's needed is to run:

```
sslm-mgr -renew
```

which will check get new certs, if it's time to renew. A couple of hours later make those certs live by doing:

```
sslm-mgr -roll
```

7.1.1 sslm-mgr options

Has 2 modes - a *normal* mode and a developer or *dev* mode. In both cases, the groups and services are read from the same config files. Some values *can* be overridden from the command line.

To specify a group and service(s) on the command line use the format:

```
sslm-mgr ... <group-name>:<service_1>,<service_2>,...
```

For example, for a domain with multiple services, you can limit to one or two services using:

```
sslm-mgr -s example.com:mail-ec
sslm-mgr -s example.com:mail-ec,mail-rsa
```

Help command for *sslm-mgr*

```
Command line options over-ride settings taken from the config file.
For example any groups/services given on the command line.


usage: /usr/bin/sslm-mgr [.. options ... } [grps_svcs ...]

positional arguments:
  grps_svcs                 groups/services: grp1:[sv1, sv2,...] grp2:[ALL] ...

options:
    --help                    show help message
    --clean-all               Clean up all grps/svcs not just active domains
    --clean-keep   <Num>      Clean database keeping newest Num
    --debug                   debug mode : print but dont do it
    --dns-refresh             dns: Use script to sign zones & restart primary
                              See config dns.restart_cmd
    --force                   Forces renew / roll / prod check
    --dry-run                 Letsencrypt --dry-run
    --reuse                   Reuse curr key with renew.tlsa unchanged if
                              using selector=1 (pubkey)
    --renew                   Renew keys/csr/cert keep in next
                              See also config setting renew_expire_days
    --roll                    Roll Phase : Make next be the new curr then
                               copy to production
                              Refresh dns if needed
    --min-roll-mins <minutes> Only roll if next is older than this
                              See config min_roll_mins.
    --status                  Display cert status. With --verb shows more info
    --test                    Letsencrypt --test-cert
    --verb                    More verbose output
```

When more control is needed then *dev* mode offers the above commands plus few more. To see developer help:

```
usage: /usr/bin/sslm-mgr (as above plus) [ ... dev options ...] [grps_svcs ...]

SSL Manager Dev Mode

options:
    ... same as above plus:
```

(continues on next page)

(continued from previous page)

```
    --new-cert                Make new next/cert
    --copy-csr                Copy curr key to next (used by --reuse)
    --new-csr                 Make next CSR
    --force-server-restarts   Forces server restarts even if not needed
    --new-keys                Make next new keys
    --next-to-curr            Move next to curr
```

7.2 Configuration Files

Sample configs are show in Appendix *Appendix* and the files are provided in *examples/conf.d* directory.

When setting for the first time, up its may be helpful to start by creating a self signed CA and use that. When you're ready, change the signing CA to Letsencrypt (LE) and run against the LE test-cert server by using

```
sslm-mgr --test
```

You may also use the Letsencrypt *–dry-run* option.

Once that is working then you switch to the normal LE server by dropping the test option.

Config files are located in *conf.d*. There are 2 shared configs and one config for each group/service. Service configs files reside under their *group* directory.

The top level configs are *ssl-mgr.conf* and *ca-info.conf* which are used for all groups and services.

ssl-mgr.conf is the main config file and we'll go over it in detail. It contains the list of domains and their services.

Groups can be marked active (active = true). In inactive group is not processed. *sslm-mgr* can optionally have 1 or more groups and services provided on the command line. Any groups/services specified on the command line are the only ones that are run.

ca-info.conf is a list of available CAs. Each CA name can be referenced in the service configs to determine which CA is to provide the signed cert.

As explained earlier, there are 2 kinds of groups: *CA*s and *Apex Domain* groups. The *CA* group is for self created CAs and an *Apex Group* contains 1 more services from which a key/certificate pair are to be created.

It is convenient to name services for their purpose (mail, web etc) and also for any characteristics of the certificate, such key type (RSA, Elliptic Curve) and sometimes by the CA name as well.

Each service config resides under its group:

```
conf.d/<group>/<service>
```

This file describes the organization and details for this one service. It specifies which CA is to sign the certificate as well as any DANE TLS[1] info needed to generate TLSA records.

N.B. Each service is to be signed by the designated CA.

If you want 2 certs signed by 2 different CAs, e.g. both self and letsencrypt, then each would have it's own distinct service name and config file.

E.g. mail-self and mail-le. For each domain, the TLSA records for all services are aggregated into a single file, tlsa.rr which can be included by the DNS server apex zone file.

[1] TLSA https://datatracker.ietf.org/doc/html/rfc6698

N.B.
Letsencrypt signing the same CSR counts towards their limits independent of validation method used (http-01 or dns-01).

7.2.1 Service Config

Info for each service to create it's cert. Each domain may have separate certs for different services (mail, web, etc). Each service must therefore have it's own unique config file. Its good practice to use separate certs for each different use cases, to help mitigate any impact of key related security issues.

Each config provides:

- Organization info (CN, O, OU, SAN_Names, ...)
- name, org, service (mail, web etc)
- Which CA should will be requested to sign this cert + validation method). Self signed dont need a validation method. + Letsencrypt, for example, allows http-01 and dns-01 as validation methods.
- DANE TLS info - list of (port, usage, selector, match) - e.g. (25,3,1,1)
- Key type for the public/private key pair

7.3 Output

All generated data is kepy in a dated directory under the *db* dir and links are provided for *curr* and *next*

- curr -> db/<date-time>
- next -> db/<date-time>

After a cert has been successfully generated, each dir will contain :

File	What
privkey.pem	private key
csr.pem	certificate signing request
cert.pem	certificate
chain.pem	root + intermediate CA cert
fullchain.pem	cert + chain
bundle.pem	privkey + fullchain
tlsa.rr	Dane TLSA records for this service (if any)
info	Contains date/time when next was rolled to curr (curr only)

The bundle.pem file, which has the priv key, is preferred by postfix to provide atomic udpate and avoid a potential race during updates. Such a race is conveivable if key and cert are read from separate files.

In addition there are the acme challenge files. The *ssl-mgr.conf* file specifies where these files are to be written.

7.3.1 DNS-01 Validation

For dns-01 the location is specified as a directory:

```
[dns]
    acme_dir = '...'
```

The acme challenges will be saved into a file under <*acme_dir*> with apex domain name as suffix:

```
<acme_dir>/acme-challenge.<apex_domain>
```

The format of the DNS resource record is per RFC 8555[2] spec. The challenge file should be included by the DNS zone file for that apex domain. Once the challenge session is complete, the file will be replaced by an empty file, which ensures that there are no errors including it in the domain zone file.

7.3.2 HTTP-01 Validation

For http-01 validation the location is specified by *server_dir* directory:

```
[web]
    server_dir = '...'
```

The individual challenge files, one per (sub)domain will be saved in a file following RFC 8555[3] spec:

```
<server_dir>/<apex_domain>/.well-known/acme-challenge/<token>
```

If the web server is not local then ssh will be used to push the file to the remote server.

N.B. In all cases please ensure that the process has appropriate write permissions.

7.3.3 DANE-TLSA DNS File

If DANE is active for any service, then the TLSA records for that service are saved under *certs/<apex-domain>/<service>/tlsa.rr*. All service TLSA records for each apex domain are then aggregated under *certs/<apex-domain>/tlsa.<apex-domain>*.

The apex domain TLSA file is then copied to one or more directories specified in the *[dns]* section of *ssl-mgr.conf*.

This file is what can be included in the DNS zone file for that apex domain.

```
[dns]
    ...
    tlsa_dirs = [<tlsa_1>, <tlsa_2>, ...]
```

Each directory, *<tlsa_1>*, *<tlsa_2>* etc, will be populated with one file per apex_domain where each file contains the TLSA records for that domain. Each apex domain tlsa file will be named:

```
tlsa.<apex_domain>
```

Each file should be included by the DNS zone file for that apex domain.

N.B. Mail server needs a TLSA record for each key/certificate is used. If, for example, postfix is set up to use either *RSA* or *EC* certs, then you **must** provide a TLSA record for both of them. And there must be record for the apex domain as well as every MX host.

We determine the MX hosts from DNS lookup of the apex domain.

7.4 Service Config for TLSA

The service config allows DANE to be specified.

The input field takes the form of a list, one item per port:

[2] DNS-01 Acme Challenge URI -> https://datatracker.ietf.org/doc/html/rfc8555#section-8.4
[3] HTTP-01 Acme Challenge URI -> https://datatracker.ietf.org/doc/html/rfc8555#section-8.3

```
dane_tls = [[25, 'tcp', 3, 1, 1], [...], ...]
```

Each item has port (25 here), the network protocol (tcp) along with *usage* (3), *selector* (1) and *hash_type* (also 1).

You should use (3,1,1).

The dane records normally contain the current TLSA records. During rollover they contain both current and next ones, and after rollover completes, and next becomes current then we're back to the normal case with only current TLSA records.

Each apex domain has it's own file of TLSA records, *tlsa.<apex_domain>*.

The *ssl-mgr.conf* DNS section also specifies where these DNS TLSA record files should be copied to - so that the DNS tools can include them in the apex domain zone file.

The best way to handle the dane resource records is by using $INCLUDE in dns zone file to picks up *tlsa.<apex_domain>* file.

DNS server is refreshed (i.e. zone files signed and primary server is restarted) whenenever a dane tlsa file changes.

The TLSA records change when the private key is updated (leading to change in the hash itself) or when the dane-info is changed (e.g. change of ports or other dane info). It certainly changes after a *renew* builds new keys/certs in *next* and after *roll* when the new *curr* is updated.

For doing rollover properly, order is important.

```
curr □  curr + next □   DNS
```

After 2xTTL or longer:

```
next □  curr □   update mail server □   refresh DNS
```

sslm-mgr takes care of this.

While it is true that reusing a key, means not having to deal with key rolloever as often, that only helps when doing things manually. And in fact even doing it manually, doing things less frequently may mean mistakes are more likely. There is also a small security reduction obviously in reusing a key.

When things are automated, as here with *sslm-mgr* taking care of everything, then there is little benefit to key reuse. So we support it, but we recommend just renew and roll and all will be fine :)

7.5 Certbot: Notes

A few notes on certbot and how we're using it.

In addition to the database directory (*db*) there is also a *cb* dir which is provided to certbot. Certbot uses to to keep letsencrypt accounts. Each group-service has its own everything - this includes it's own certbot *cb* and thus separately registered LE (Letsencrypt) account for each service.

We are using cerbot in manual mode. This gives us a lot of control and allows us to use our own generated CSR as well as to specify where the resulting cert and chain files get stored.

When sending a CSR with apex domain plus sub-domains, each (sub)domain gets a challenge and each challenge must be validated by LE before cert is issued. Challenges can be validated by acme http-01 or dns-01. Wildcard sub-domains (*.example.com) can only be validated using dns-01.

Certbot sends each challenge to a *hook* program. The *hook* program is called once per challenge. Information about the challenge and which sub-domain are passed to the *hook* program in environment variables. Env variables also tell the program how many more challenges remain to be sent. Once all the challenges have been delivered - and

only after the *hook* program returns - LE will then seek to validate all of the acme challenges, whether http or dns validation is being used.

This is actually really good - it means that we can push all the challenges out - and wait for every DNS authoritative name server to have the TXT records before allowing the hook to return once it has every acme challenge.

In older versions of certbot, validation took place after each sub-domain challenge, and for DNS that meant dns refresh - wait for NS to udpate - LE checks and sends next challenge. This could potentially very long wait times - I read of some folks waiting many hours. Now with the new way as described above, whether DNS or HTTP challenge, it takes only seconds or minutes.

It seems to me that LE checks directly with each authoritative NS, which is the most efficient way to check - rather than waiting on some random recursive server to get updated.

7.6 sslm-mgr application

7.6.1 Usage

To run - go to terminal and use :

```
sslm-mgr --help
```

7.6.2 Config File Location

The configuration file for ssl-mgr is chosen by checking for as the first directory containing a *conf.d* directory from the list of *topdir* directories:

```
<SSL_MGR_TOPDIR>
./
/etc/ssl-mgr/
```

Where the directories are checked in order and *<SSL_MGR_TOPDIR>* is an environment variable that can be set to take preference.

The config files are located in *topdir/conf.d/* and certificates, key files and so on are saved under *topdir/certs*.

For example if there environment variable is not set and the directory *./conf.d* exists, then it becomes the top level directory. And all configuration files reside under *./conf.d*.

7.7 Log Files

Logs are found in the log directory specified by the global config variable:

```
[globals]
    ...
    logdir = 'xxx'
```

There are 3 kinds of log files in the log directory.

- *<logdir>/sslm*: General application log
- *<logdir>/cbot*: Application log while interacting with letsencrypt via certbot.
- *<logdir>/letsencrypt/letsencrypt.log.<N>*: Letsencrypt log provided by cerbot.

CHAPTER

EIGHT

APPENDIX

8.1 Dealing with Possible Problems

Once the configuration files are set up it can be helpful to start with self-signed CA root certificate and a local CA intermediate certificate (signed by that root cert). Use the local intermediate cert to sign your application certificates.

Once this is all working then try using Letsencrypt CA. Upon successful completion the *certs* directory holds all outputs including historical data (older certs and so on).

A subset of the output data is then copied to the production directory. It may also be copied to remote servers if configs request that. After certs are updated and copied then the list of programs to run specified in the *post_copy_cmd* config variable will be run.

If there is a problem for some reason, then updating production will be avoided to minimize any production impact. It is conceivable that after some error conditions, the production cert directory could get out of sync. Or a server reboot while production certs are being updated.

On any subsequent run after experiencing some error condition, When *sslm-mgr* starts, it detects if any production cert files are out of sync. If so, a warning is issued and production cert dirs are updated and servers are restarted.

Manual intervention should not be required but if you need it for some reason, the *dev* option gives ability to force production resync and to restart servers.

This can be done using dev options:

```
sslm-mgr dev --force --certs-to-prod
```

to bring production back in sync. To restart the servers use:

```
sslm-mgr dev --force-server-restarts
```

And you may alos want to renew the certs:

```
sslm-mgr -renew
```

and wait the usual 2-3 hours and roll as usual:

```
sslm-mgr -roll
```

8.2 Self Signed CA

The *examples/ca-self* directory has sample how to do this. The CA has a self-signed root certificate (*my-root*) along with an intermediate certificate (*my-int*) which is signed by the root cert. Other certs are then signed by the intermediate certificate.

The 2 public CA certs then need to be added to the linux certificate trust store. To do this copy each cert as below and update the trust store:

```
cp certs/ca/my-root/curr/cert.pem /
 ↪etc/ca-certificates/trust-source/anchors/my-root.pem
cp certs/ca/my-int/curr/cert.pem /
 ↪etc/ca-certificates/trust-source/anchors/my-int.pem
update-ca-trust
```

Since browsers do not typically use the system certificate store the same certs will need to be imported into each browser. This can be dont manually in the GUI or using *certutil* provided by the *nss* package. Modern browsers typically keep the certificates in a file called *cert9.db* which can be updated using for example something like this (untested):

```
cert9='<path-to>/cert9.db'
cdir=$(dirname $cert9)
certutil -A -n "my-int" -t "TC,C,TC" -i xxx/my-int/curr/cert.pem -d sql:$cdir
```

Please see *certutil* man pages for more info.

8.3 Sample Cron File

```
#
# Renew certs
#  - certs renew (check) every Tue afternoon and roll 3 hours later
#
30 14 * * 2 root /usr/bin/sslm-mgr -renew
30 17 * * 2 root /usr/bin/sslm-mgr -roll

#
# update dh parms:
# will update if existing file is older than min age.
# The default min age is 120 days. Use -a to change min age.
#
30 2 5 * 2 root /usr/bin/sslm-dhparm -s /etc/ssl-mgr/prod-certs
```

8.4 Config ca-info.conf

```
[le-dns]    # Used to sign client certs
    ca_desc = 'Letsencrypt: dns-01 validation'
    ca_type = 'certbot'
    ca_validation = 'dns-01'

[le-http]    # Used to sign client certs
    ca_desc = 'Letsencrypt: http-01 validation'
    ca_type = 'certbot'
    ca_validation = 'http-01'

[my-root] # To sign our own intermediate 'sub' certs
    ca_desc = 'My Self signed root : EC signs my intermediate certs'
    ca_type = 'self'
```

(continues on next page)

(continued from previous page)

```
[my-sub]  # Used to sign client certs
    ca_desc = 'My intermediate : EC signs client certs'
    ca_type = 'local'
```

8.5 Config ssl-mgr.conf

```
[globals]
    verb = true
    sslm_auth_hook = '/usr/lib/ssl-mgr/sslm-auth-hook'      # For certbot
    prod_cert_dir = '/etc/ssl-mgr/prod-certs'
    logdir = '/var/log/ssl-mgr/ssl-mgr/Logs'

    clean_keep = 10
    min_roll_mins = 90

    #
    # Letsencrypt profiles: classic, tlsserver, shortlived
    # NB classic is going away.
    #
    preferred_acme_profile = 'tlsserver'

    dns_check_delay = 240
    dns_xtra_ns = ['1.1.1.1', '8.8.8.8', '9.9.9.9', '208.67.222.222']

    post_copy_cmd = [['example.com', '/etc/ssl-mgr/tools/update-permissions'],
                     ['voip.example.com', '/etc/ssl-mgr/tools/voip-checker']
                     ]

[renew_info]
    # target times to expiration when to renew
    target_90 = 30.0        # was renew_expire_days in globals section
    target_60 = 20.0
    target_45 = 10.0
    target_10 = 5.0
    target_6 = 2.0
    target_2 = 1.0
    target_1 = 0.5

    # random variability days (0 means no variability)
    rand_adj_90 = 3.0       # was renew_expire_days_spread in globals section
    rand_adj_60 = 0.0
    rand_adj_45 = 0.0
    rand_adj_10 = 0.0
    rand_adj_6 = 0.0
    rand_adj_2 = 0.0
    rand_adj_1 = 0.0


#
```

(continues on next page)

(continued from previous page)

```
# Groups & Services
#
[[groups]]
    active=true
    domain='example.net'
    services=['web-ec']

[[groups]]
    active=true
    domain = 'example.com'
    services = ['mail-ec', 'mail-rsa', 'web-ec']

[[groups]]
    active=true
    domain = 'ca'
    services = ['my-root', 'my-sub']

#
# DNS primary provides authorized NS (name servers) and MX hosts of apex_domain
# Must have at least one for acme dns-01
#
[[dns_primary]]
    domain = 'default'
    server = '10.1.2.3'
    port = 11153

[[dns_primary]]
    domain = 'example.com'
    server = '10.1.2.3'
    port = 11153

#
# Servers
#
[dns]
    restart_cmd = '/etc/dns_tools/scripts/resign.sh'
    acme_dir = '/etc/dns_tool/dns/external/staging/zones/include-acme'
    tlsa_dirs = ['/etc/dns_tool/internal/staging/zones/include-tlsa',
                 '/etc/dns_tool/external/staging/zones/include-tlsa',
                 ]

    # restart trigger when dns (TLSA) zones have changed.
    depends = ['dns']

[smtp]
    servers = ['smtp1.internal.example.com', 'smtp2.internal.example.com']
    # If using sni_maps
    #restart_cmd = ['/usr/bin/postmap -F lmdb:/etc/postfix/sni_maps', '/
↪usr/bin/postfix reload']
    restart_cmd = '/usr/bin/postfix reload'
    svc_depends = [['example.com', ['mail-rsa', 'mail-ec']]]
    depends = ['dns']
```

(continues on next page)

(continued from previous page)

```
[imap]
    servers = ['imap.internal.example.com']
    restart_cmd = '/usr/bin/systemctl restart dovecot'
    svc_depends = [['example.com', ['mail-rsa', 'mail-ec']]]

[web]
    servers = ['web.internal.example.com']
    restart_cmd = '/usr/bin/systemctl reload nginx'
    server_dir = '/srv/http/Sites'                  # Used for acme http-01
↪validation
    svc_depends = [['any', ['web-ec']]]

[other]
    # these servers get copies of certs
    servers = ['backup.internal.example.com', 'voip.internal.example.com']
    restart_cmd = ''
```

8.6 Config Service : example.com/mail-ec

```
#
# example.com : mail-ec
#
name = 'Example.com Mail'
group = 'example.com'
service = 'mail-ec'

#signing_ca = 'my-sub'
#signing_ca = 'le-http'
signing_ca = 'le-dns'
renew_expire_days = 30

# Include tls.example.com in zone file to use
#  => [[port, proto, usage, selector, match], ...]
# port 25 uses MX other ports use all SAN hostsnames.
dane_tls = [[25, 'tcp', 3, 1, 1]]

[KeyOpts]
    ktype = 'ec'
    ec_algo = 'secp384r1'

[X509]
    # X509Name details
    CN = 'example.com'
    O = 'Example Company'
    OU = 'IT Mail'
    L = ''
    ST = ''
    C = 'US'
    email = 'hostmaster@example.com'    # required to register with letsencrypt
```

(continues on next page)

(continued from previous page)

```
    sans = ['example.com', 'smtp.example.com', 'imap.example.com',
↪'mail.example.com']
```

8.7 Directory tree structure

Directory Structure. By default we only use EC keys, can add RSA if required. We use 'ec' as a label to keep things clear and allow easy way to change to new key types (RSA or other).

Input:

```
conf.d/
    ssl-mgr.conf
    ca-info.conf

    example.com/
        mail-ec
        mail-rsa
        web-ec

    example.net/
        web-ec

    ca/
        my-root
        my-sub
    ...
```

Output - Final Production Certs:

```
prod-certs/
    example.com/
        tlsa.example.com

        dh/
            dh2048.pem
            dh4096.pem
            dhparam.pem -> dh4096.pem
            ...
        mail-ec/
            curr/
                privkey.pem
                csr.pem
                chain.pem
                fullchain.pem
                cert.pem
                bundle.pem
                tlsa.rr
                info
        web-ec/
            ...
        ...
```

Output - Internal Data

```
certs/
    example.com/
        tlsa.example.com

        mail-ec/
            curr -> db/date1
            next -> db/date2

            db/date1/
                csr.pem
                privkey.pem
                cert.pem
                chain.pem
                fullchain.pem
                bundle.pem
                tlsa.rr
            cb/
                [files used by cerbot]

        web-ec/
            curr -> db/date1
            next -> db/date2

            db/date1/
                ...
            cb/
                [files used by cerbot]

        .. other services

    example.net/
        ...
```

8.8 Installation

Available on

- Github
- Archlinux AUR

On Arch you can build using the provided PKGBUILD in the packaging directory or from the AUR. To build manually, clone the repo and :

```
./scripts/do-build
./scripts/do-install <destination-directory>
```

8.9 Dependencies

- Run Time :

Package	Comment
python	3.14 or later
cryptography	
dateutil	
dnspython	
lockmgr	Ensures 1 app runs at a time
pyconcurrent	
bash	
tomli-w	

- Optional

dns_tools

- Building Package:

Package	Comment
git	
meson	
meson-python	
rsync	